

1. Self-Steering Medical Diagnostic Assistant

(Strong + practical + publishable)

Since tum already diagnostic assistant explore kar chuki ho, this is naturally strong for you.

Core Idea

Instead of directly generating diagnosis:

Planner:

- decides reasoning workflow
- asks which symptoms matter
- selects possible diseases
- decides which tests/questions to ask next

Follower:

- generates hypotheses
- evaluates symptom consistency
- produces explanations

Inference engine:

- explores multiple diagnosis paths
 - prunes unlikely diseases
 - reallocates compute to promising diagnoses
-

Example

Input:

fever + chest pain + cough

Planner:

- respiratory branch explore karo
- infection probability evaluate karo
- pneumonia vs TB vs viral infection compare karo

Follower:

- generates structured reasoning paths
-

Research Contribution

Possible novelty:

- adaptive medical reasoning

- confidence-aware branching
 - multi-stage diagnosis search
 - verifier-guided clinical reasoning
-

Why strong?

Because:

- healthcare + agentic AI hot topic
 - structured reasoning very important
 - small models + orchestration useful
 - good publishability potential
-

2. DISCIPL for Mathematical Reasoning

(Very research-oriented)

Problem

LLMs:

- arithmetic mistakes karte hain
 - logic jump kar dete hain
 - incorrect intermediate reasoning dete hain
-

Your Idea

Planner:

- multiple solution strategies generate kare
- equation solving steps divide kare

Follower:

- stepwise solve kare

Verifier:

- symbolic checker use karo
- equation consistency verify karo

Inference:

- incorrect branches remove
- correct paths expand

Advanced Extensions

- backtracking
- iterative correction
- theorem verification
- symbolic algebra integration

Why interesting?

Current frontier research area:

- test-time scaling
- reasoning search
- verifier-guided math reasoning

Very strong publication direction.

3. Adaptive Compute Allocation for Small LMs

(More systems/research focused)

This is VERY aligned with DISCIPL future work.

Current limitation

DISCIPL mostly fixed particle budget use karta hai.

Example:

- every branch gets same compute initially

But real reasoning mein:

- easy branches ko kam compute chahiye
- hard branches ko zyada

Your Idea

Dynamic scheduler banao:

- promising branches pe more particles
- weak branches prune
- uncertainty-based compute allocation

Example

If:

- one reasoning path highly coherent hai
- another random lag raha hai

System:

- good branch pe more search kare
-

Possible Contributions

- uncertainty-aware inference
 - adaptive SMC
 - dynamic branching policy
 - compute-efficient reasoning
-

Why good?

Strong systems + inference engineering topic.

Could become:

- workshop paper
 - efficient AI systems paper
-

4. Multi-Agent Self-Steering Reasoning System

(Very trendy area)

Current DISCIPL

Only:

- Planner
 - Follower
-

Your Extension

Add:

- Critic agent
 - Verifier agent
 - Memory agent
 - Retrieval agent
-

Example Workflow

Planner:

solve ka strategy decide karo

Follower:

answer generate karo

Critic:

flaws detect karo

Verifier:

constraints check karo

Memory:

previous attempts remember karo

Research Question

Does specialized multi-agent orchestration improve reasoning quality?

Possible Domains

- coding
 - legal drafting
 - planning
 - research assistance
 - medical reasoning
-

Why strong?

Agentic AI is one of the hottest current areas.

Very aligned with:

- AutoGen
 - CrewAI
 - Devin-like systems
 - reasoning orchestration
-

5. Iterative Editing + Self-Repair DISCIPL

(My favorite research-style idea)

This directly extends paper limitations section.

Problem

Current DISCIPL mostly:

- generate → evaluate

But humans:

- draft
 - revise
 - edit
 - improve
-

Your Idea

Instead of one-pass generation:

1. Generate draft
 2. Detect problems
 3. Edit partial sections
 4. Rescore
 5. Repeat
-

Example

Task:

write constrained paragraph

System:

- first draft
 - detect character count issue
 - repair only problematic sentence
 - maintain fluency
-

Research Angle

Compare:

- one-shot generation
vs
 - iterative repair
-

Possible Improvements

- local editing
 - semantic preservation
 - partial regeneration
 - correction memory
-

6. Constraint-Aware Coding Assistant

(Industry useful)

Problem

LLMs generate code that:

- violates constraints
 - misses APIs
 - fails tests
-

Your Idea

Planner:

- decomposes coding task
- generates verification steps

Follower:

- generates code

Verifier:

- runs tests/static checks

Inference:

- explore multiple implementations
-

Research Direction

“Structured inference for reliable code generation”

Very practical.

7. Budget-Aware Planning Agent

(Simple but publishable)

Example Tasks

- travel planning
- shopping assistant
- event planning
- resource allocation

Constraints:

- money
 - time
 - distance
 - preferences
-

Why useful?

Perfect for:

- SMC
- pruning
- dynamic scoring
- self-hinting

Easy to prototype.

8. Self-Steering Research Assistant

(Very modern)

Planner:

- decides search strategy
- decomposes research question

Follower:

- summarizes papers
- extracts findings

Critic:

- checks factual consistency

Could combine:

- retrieval
- planning

- structured reasoning
-

My recommendation for YOU specifically

Based on:

- your interests
- BTP context
- current understanding

Best 3 options:

BEST OPTION

Self-Steering Medical Diagnostic Assistant

Why?

- aligned with your previous work
 - practical
 - research-worthy
 - lots of extensions possible
-

SECOND BEST

Iterative Self-Repair DISCIPL

Why?

- directly extends paper
 - clean research contribution
 - feasible
-

THIRD BEST

Adaptive Compute Allocation

Why?

- deep systems/research angle
 - unique contribution potential
 - inference engineering focus
-

Sir, I went through the DISCIPL paper and the accompanying Sprinklr blog.

From what I understood, the core idea of the paper is not just improving prompting, but changing the way inference itself is structured for language models.

Traditionally, most LLM reasoning methods rely on autoregressive generation — meaning the model generates one token after another in a long chain-of-thought. This is flexible, but it is slow, expensive, difficult to parallelize, and often unreliable for tasks involving strict constraints like counting, formatting, budgeting, or positional rules.

The paper argues that instead of forcing a model to do all reasoning inside one long generation stream, we can externalize and structure inference-time computation.

Existing structured inference methods like:

- tree search,
- Sequential Monte Carlo (SMC),
- beam search,
- rejection sampling

already try to do this by maintaining multiple candidate generations and exploring them in parallel.

However, these methods usually require humans to manually engineer:

- search strategies,
- scoring functions,
- verifiers,
- constraint handlers,
- heuristics.

The key contribution of DISCIPL is:

the LM itself dynamically writes the inference procedure.

So instead of humans designing the reasoning/search algorithm for every task, the model generates a task-specific inference program on the fly.

The architecture has two roles:

1. Planner LM
2. Follower LM

The Planner is the “meta-reasoner.”

It does not directly solve the task.

Instead, it:

- interprets the task,
- decides how reasoning should happen,
- decomposes the task into steps,
- defines constraints,
- decides how candidates should be scored,
- and generates an inference program.

The Follower is a smaller LM that executes this program.

The generated program is written in LLAMPPL, which is a probabilistic programming framework for language models.

The program can:

- sample tokens,
- force constraints,
- score partial generations,
- maintain multiple candidates,
- reject bad generations,
- dynamically allocate compute,
- and guide search.

A very important idea in the paper is:
reasoning is decomposed into planning + execution.

So:

- Planner decides the reasoning strategy
- Follower performs the actual generation/search

This allows small models to perform much better than their original standalone capability.

The paper evaluates DISCIPL mainly on:

1. COLLIE → constrained generation benchmark
2. PUZZLES → naturalistic structured reasoning tasks

The constrained generation tasks are interesting because normal LLMs fail badly at things like:

- exact character counts,
- positional constraints,
- budget constraints,
- structured formatting,
- syllable constraints.

DISCIPL improves this significantly because it does not rely on a single generation path.

The paper studies three main inference methods:

1. Rejection Sampling (RS)
Generate complete outputs and reject invalid ones.
2. Importance Sampling (IS)
Generate multiple candidates fully, score them, and prefer high-scoring outputs.
3. Sequential Monte Carlo (SMC)
The most interesting method.

SMC maintains multiple partial generations called particles.

After each generation step:

- good particles get duplicated,
- bad particles are removed,
- compute is dynamically shifted toward promising branches.

This gives:

- better resource allocation,
- higher validity,
- and better coherency.

The paper also introduces several “self-steering” patterns:

- Token masking
Enforce constraints during generation itself.
- Self-hinting
Planner dynamically injects intermediate symbolic information into the Follower context.
- Self-checking
Programs verify outputs after generation.

One thing I found especially interesting is that DISCIPL treats inference engineering itself as something automatable.

Normally, humans manually design:

- beam search,
- decoding tricks,
- search heuristics,
- constraint handlers.

DISCIPL instead allows the LM to generate these procedures dynamically.

So the model is not just generating text anymore — it is generating reasoning algorithms.

Another important insight from the experiments was:

small models like Llama-1B become significantly more capable when orchestrated properly through structured inference.

In some constrained tasks, DISCIPL with a small Follower even approaches strong reasoning models like o1.

The paper also suggests future directions like:

- iterative editing,
- backtracking,
- MCTS-style search,
- mathematical reasoning,
- recursive self-improvement,

- and self-steering using the same model as both Planner and Follower.

From a systems perspective, I think the paper is really about:

“using LMs to control compute allocation and reasoning structure dynamically at inference time.”

Instead of:

“one giant model thinks harder,”

the idea becomes:

“many smaller reasoning paths are orchestrated intelligently.”